# SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

## HIGH PERFORMANCE COMPUTING (BCSE414L)

**Review 1: High Performance Computing Course Project**

### 1. Title

# Skew-Aware Adaptive Parallel Near-Duplicate Detection on Multicore CPUs

### Group Details


| | |
|---|---|
| **Group Number:** | 29 |
| **Submitted By:** | Lakshmi Sarupa Venkadesh |
| **Register No.:** | 23BCE0463 |
| **Course Code:** | BCSE414L |
| **Faculty Guide:** | Vimala Devi K. |
| **Academic Year:** | 2026–27 |

# Table of Contents

**1. Title** . . . . . **1**

**Group Details** . . . . . **1**

**2. Abstract** . . . . . **3**

**3. Introduction** . . . . . **3**

3.1 Background and Motivation . . . . . 3

3.2 The Pairwise Comparison Bottleneck . . . . . 3

3.3 The Workload Skew Problem on Multicore CPUs . . . . . 3

3.4 Project Objectives . . . . . 3

**4. Literature Survey / Existing Work** . . . . . **4**

**5. Proposed Methodology with Novelty** . . . . . **5**

5.1 Dataset Generation . . . . . 5

5.2 Document Fingerprinting . . . . . 5

5.3 Candidate Generation . . . . . 6

5.4 Similarity Verification . . . . . 6

5.5 Sequential Baseline . . . . . 6

5.6 Standard Parallel Implementation (OpenMP) . . . . . 6

5.7 Proposed Workload-Aware Adaptive Scheduling . . . . . 6

5.8 Experimental Evaluation . . . . . 7

5.9 Novelty in the Proposed Work . . . . . 7

5.10 Deliverables & Expected Outcomes . . . . . 8

**6. References** . . . . . **8**

## 2. Abstract

Near-duplicate detection is the problem of finding documents in a collection that have the same or almost the same text. Comparing every document with every other document takes too much time as the dataset grows. Fingerprinting methods help reduce the number of comparisons by grouping likely matches into buckets, but these buckets often end up with very uneven sizes. This creates a serious load imbalance when the work is split across CPU cores using simple parallel loops.

This project proposes a parallel near-duplicate detection system on the CPU using SimHash fingerprinting and OpenMP. The system will group documents into candidate pairs using fingerprint bands and check those pairs using Hamming distance. A single-threaded sequential version will be built first as a baseline. Standard OpenMP loop schedules (`static`, `dynamic`, and `guided`) will then be compared against a workload-aware adaptive scheduling approach that estimates the work inside each candidate bucket and splits up very large buckets across threads.

The system will be written in C++17 with OpenMP and tested on synthetic datasets with controllable skew. Experiments will vary the number of documents, the level of skew, the number of threads, and the scheduling policy. The evaluation will measure execution time, speedup, parallel efficiency, thread load balance, memory usage, and detection accuracy to see when workload-aware scheduling improves performance on a multicore CPU.

## 3. Introduction

### 3.1 Background and Motivation

Large document collections, such as web pages, digital libraries, legal archives, and medical records, often contain many duplicate or near-duplicate files caused by copying, shared templates, and minor edits [1], [4], [5]. Finding these near-duplicates quickly is important for saving storage space, improving search results, and cleaning datasets before analysis.

### 3.2 The Pairwise Comparison Bottleneck

For a collection of $N$ documents, checking every possible pair requires $\frac{N(N-1)}{2} = \mathcal{O}(N^2)$ comparisons. As $N$ gets larger, comparing all pairs directly becomes too slow. Locality-Sensitive Hashing (LSH) methods such as MinHash [1] and Charikar's SimHash [2] solve this by turning each document into a short binary fingerprint where similar documents have similar fingerprints. By splitting each fingerprint into bands, only documents that match in at least one band need to be compared as candidate pairs [4].

### 3.3 The Workload Skew Problem on Multicore CPUs

Even though banding filters out most unrelated pairs, real document collections rarely spread documents evenly across hash buckets [3], [6], [7]. Common templates or frequently copied texts fall into the same buckets. Since a bucket with $k$ documents produces $\frac{k(k-1)}{2}$ pairs to check, even a moderate increase in bucket size causes a huge jump in comparison work. When standard OpenMP schedules (`static`, `dynamic`, or `guided`) assign whole buckets to threads, one thread can get stuck working on a giant bucket while other cores finish early and wait, or dynamic scheduling can add unnecessary overhead on tiny buckets.

### 3.4 Project Objectives

The main goals of this project are to:

1. Build a synthetic dataset generator that can create both balanced and skewed duplicate groups.
2. Implement a sequential baseline pipeline using SimHash fingerprinting, band-based candidate generation, and Hamming-distance verification.
3. Parallelize the pipeline in C++17 using OpenMP and test standard scheduling modes (`static`, `dynamic`, and `guided`).
4. Design and implement a **workload-aware adaptive scheduling method** that estimates the work in each bucket before execution and breaks very large buckets into smaller parallel tasks.
5. Measure execution time, speedup, parallel efficiency, and load balance across different dataset sizes, skew levels, and thread counts.

## 4. Literature Survey / Existing Work

### 1. On the Resemblance and Containment of Documents

> ***Broder** [1] introduced the mathematical model for measuring document similarity and containment using MinHash and random sampling. This paper showed how similar documents can be found without comparing every pair of documents directly.*

### 2. Similarity Estimation Techniques from Rounding Algorithms

> ***Charikar** [2] introduced locality-sensitive hashing based on random hyperplane rounding, known as **SimHash**. This work showed that the similarity between two high-dimensional feature vectors can be estimated from the Hamming distance between their compact binary fingerprints.*

### 3. LSH Forest: Self-Tuning Indexes for Similarity Search

> ***Bawa, Condie, and Ganesan** [3] proposed the LSH Forest index to handle different similarity thresholds without manual tuning, and showed how uneven data distributions can slow down similarity searches.*

### 4. Probabilistic Near-Duplicate Detection Using SimHash

> ***Sood and Loguinov** [4] analyzed how SimHash performs on large web datasets. They showed that while SimHash is very good at filtering out unrelated documents, generating and checking the remaining candidate pairs still takes significant compute time.*

### 5. Near-Duplicate Detection in an Academic Digital Library

> ***Williams and Giles** [5] tested SimHash and shingle-based duplicate detection on the CiteSeerX digital library, comparing fingerprint size, filtering speed, and detection accuracy.*

### 6. A Scalable Parallel Deduplication Algorithm

> ***Santos et al.** [6] built parallel deduplication algorithms on multicore CPUs and showed that uneven candidate cluster sizes cause threads to become unbalanced unless the workload is split carefully.*

### 7. Efficient String Similarity Join in Multi-Core and Distributed Systems

> ***Yan et al.*** *[7] studied parallel string similarity joins on multicore and distributed systems, showing that splitting data based on item frequency helps avoid bottlenecks from skewed inputs.*

### 8. Parallel Loop Scheduling for High Performance Computers

> ***Yue and Lilja*** *[8] compared static, dynamic, and guided loop scheduling in parallel computing and showed that no single fixed schedule works best for every type of workload.*

### 9. Accelerating Content-Defined-Chunking-Based Data Deduplication by Exploiting Parallelism

> ***Xia et al.*** *[9] developed P-Dedupe, a parallel deduplication system that separates fingerprinting and verification into parallel stages with dynamic task assignment to keep multicore CPUs busy.*

### 10. Parallel Near-Duplicate Document Detection Using General-Purpose GPU

> ***Peshevski, Zdraveski, and Ristov*** *[10] built a GPU-based MinHash pipeline for near-duplicate document detection, showing that fine-grained parallelism works well for fingerprint-based similarity search.*

### 11. Automatic Near-Duplicate Content Document Detection in a Cancer Registry

> ***Niemi et al.*** *[11] used SimHash fingerprinting followed by sequence alignment to find near-duplicate medical reports in a cancer registry, showing that fast fingerprint filtering paired with exact verification works well on real text data.*

## 5. Proposed Methodology with Novelty

The system will be implemented in C++17 using OpenMP for shared-memory parallelism.

```
Input Documents —> SimHash Fingerprinting —> Band Partitioning & Candidate Generation —>
        Skew-Aware Adaptive Parallel Verification (OpenMP) —> Final Duplicate Pairs
```

### 5.1 Dataset Generation

A synthetic dataset generator will be built to create original documents, exact copies, slightly modified copies, and unrelated documents. The generator will support configurable document lengths and control how evenly or unevenly duplicates are grouped (balanced vs. skewed distributions). A fixed random seed will be used so all experiments can be repeated on the exact same data.

### 5.2 Document Fingerprinting

Each document will be broken into small word groups (shingles) and hashed into a 64-bit SimHash fingerprint. Documents with similar wording will produce fingerprints that differ in only a few bit positions.

### 5.3 Candidate Generation

To avoid checking all $O(N^2)$ document pairs, each 64-bit fingerprint will be split into fixed-size bands. Documents that have the same band value in the same band position will be placed into the same hash bucket and treated as candidate pairs. Since the same pair of documents can match in more than one band, duplicate candidate pairs will be removed before the final check.

The candidate generation step will also be checked against a brute-force all-pairs comparison on small test datasets to make sure near-duplicate pairs are not missed.

### 5.4 Similarity Verification

For each candidate pair, the system will calculate the Hamming distance between their two 64-bit fingerprints using a bitwise XOR operation and bit counting. Any pair whose Hamming distance is less than or equal to a threshold $k$ will be reported as a near-duplicate.

### 5.5 Sequential Baseline

A single-threaded sequential version of the pipeline will be implemented first. It will be used both to check that the parallel results are correct and as the baseline time ($T_1$) for calculating speedup.

### 5.6 Standard Parallel Implementation (OpenMP)

OpenMP will be used to run the pipeline across multiple CPU threads. Three built-in loop scheduling policies will be tested:

- **Static Scheduling**: Divides loop iterations into fixed chunks ahead of time. It has very low overhead, but threads can become unbalanced if some buckets are much larger than others.
- **Dynamic Scheduling**: Assigns chunks to threads as they become free. This helps balance uneven work, but adds synchronization overhead.
- **Guided Scheduling**: Starts with larger chunks and gradually uses smaller chunks to balance overhead and load distribution.

All scheduling modes will run on the exact same input documents and candidate logic so that any runtime differences come only from how the parallel work is scheduled.

### 5.7 Proposed Workload-Aware Adaptive Scheduling

For a bucket with $k$ documents, the number of pairs to process grows quadratically:

$$\text{Cost}(k) = \frac{k(k-1)}{2}$$

When bucket sizes are skewed, a few very large buckets take up most of the execution time. The proposed adaptive approach will work in four steps:

1. **Work Estimation**: Calculate the number of pairs $\frac{k(k-1)}{2}$ in each bucket before starting the parallel comparison phase.
2. **Bucket Classification**: Separate normal-sized buckets from oversized buckets based on the distribution of bucket work.
3. **Splitting Large Buckets**: Break oversized buckets into smaller sub-tasks of pairs instead of giving an entire huge bucket to a single thread.
4. **Parallel Execution**: Schedule the normal buckets and the split sub-tasks across OpenMP threads so all cores stay evenly loaded.

This adaptive approach will be tested directly against the sequential baseline and the standard OpenMP schedules.

### 5.8 Experimental Evaluation

The experiments will test different settings for:

- **Dataset Size ($N$)**: Small to larger document collections.
- **Workload Skew**: Balanced datasets vs. skewed datasets with heavy duplicate clusters.
- **Thread Count**: 1, 2, 4, 8, and the maximum available CPU threads.
- **Scheduling Policies**: Sequential, OpenMP Static, OpenMP Dynamic, OpenMP Guided, and Proposed Adaptive.
- **Chunk Sizes**: Different chunk size settings for scheduling.

**Performance Metrics:**

- **Execution Time ($T_p$)**: Runtime measured across repeated runs.
- **Speedup ($S_p$)**: $S_p = \frac{T_1}{T_p}$
- **Parallel Efficiency ($E_p$)**: $E_p = \frac{S_p}{p}$
- **Load Imbalance**: Ratio of maximum per-thread pair work to mean per-thread pair work.
- **Candidate Buffer Storage**: Estimated memory required for raw and deduplicated candidate-pair buffers based on pair counts.
- **Correctness & Recall Checks**: Checking candidate-generation recall against brute-force all-pairs comparison on small test datasets and verifying exact output parity between the parallel and sequential pipelines.

### 5.9 Novelty in the Proposed Work

Prior studies in the literature survey cover document fingerprinting [1], [2], [4], [5], similarity indexing under data skew [3], [6], [7], and loop scheduling [8] mostly as separate problems. When parallelizing band-based candidate generation with standard OpenMP loop schedules, a straightforward bucket-level loop assigns each hash bucket as a single iteration, which can struggle when bucket sizes vary widely:

1. **Using Known Bucket Cost Before Scheduling**: Standard OpenMP loop schedules do not inspect the $\frac{k(k-1)}{2}$ pair cost inside each bucket of size $k$. The proposed approach calculates this exact pair count ahead of time.
2. **Handling Heavy Skew on Multicore CPUs**: In near-duplicate detection, a few buckets can contain thousands of pairs while most buckets have very few. Splitting only the heavy buckets prevents a single slow thread from holding up the rest of the CPU cores.
3. **Combining Bucket Statistics with OpenMP Scheduling**: The scheduler will look at how uneven the bucket work is at runtime to decide how to schedule normal buckets while dynamically splitting heavy buckets.
4. **Testing When Adaptive Scheduling Helps Most**: Experiments will be run across both balanced and skewed datasets to see clearly when adaptive scheduling beats standard OpenMP policies and when simple static or guided scheduling is already enough.

### 5.10 Deliverables & Expected Outcomes

- **Course Deliverable**: A complete project report in **IEEE conference format** covering the design, implementation, hardware setup, speedup, efficiency, and load-balance results.
- **Codebase**: A modular C++17 and OpenMP codebase with a synthetic dataset generator, SimHash fingerprinting, standard and adaptive parallel schedulers, and benchmark scripts.
- **Expected Outcome**: Clear benchmark results showing how workload skew affects parallel near-duplicate detection and how much adaptive bucket splitting improves load balance and runtime.

## 6. References

[1] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, Positano, Italy, 1997, pp. 21–29, doi: 10.1109/SEQUEN.1997.666900.

[2] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th Annu. ACM Symp. Theory of Computing (STOC)*, Montreal, QC, Canada, 2002, pp. 380–388, doi: 10.1145/509907.509965.

[3] M. Bawa, T. Condie, and P. Ganesan, "LSH forest: Self-tuning indexes for similarity search," in *Proc. 14th Int. Conf. World Wide Web (WWW)*, Chiba, Japan, 2005, pp. 651–660, doi: 10.1145/1060745.1060840.

[4] S. Sood and D. Loguinov, "Probabilistic near-duplicate detection using simhash," in *Proc. 20th ACM Int. Conf. Information and Knowledge Management (CIKM)*, Glasgow, UK, 2011, pp. 1117–1126, doi: 10.1145/2063576.2063737.

[5] K. Williams and C. L. Giles, "Near duplicate detection in an academic digital library," in *Proc. ACM Symp. Document Engineering (DocEng)*, Florence, Italy, 2013, pp. 91–94, doi: 10.1145/2494266.2494312.

[6] W. Santos, T. Teixeira, C. Machado, W. Meira Jr., A. S. da Silva, R. Ferreira, and D. Guedes, "A Scalable Parallel Deduplication Algorithm," in *Proc. 19th Int. Symp. Computer Architecture and High Performance Computing (SBAC-PAD)*, Gramado, Brazil, 2007, pp. 79–86, doi: 10.1109/SBAC-PAD.2007.32.

[7] C. Yan, X. Zhao, Q. Zhang, and Y. Huang, "Efficient string similarity join in multi-core and distributed systems," *PLoS ONE*, vol. 12, no. 3, Art. no. e0172526, Mar. 2017, doi: 10.1371/journal.pone.0172526.

[8] K. K. Yue and D. J. Lilja, "Parallel loop scheduling for high performance computers," *Advances in Parallel Computing*, Elsevier, vol. 10, pp. 243–264, 1995, doi: 10.1016/S0927-5452(06)80016-X.

[9] W. Xia, D. Feng, H. Jiang, Y. Zhang, V. Chang, and X. Zou, "Accelerating content-defined-chunking based data deduplication by exploiting parallelism," *Future Generation Computer Systems*, vol. 98, pp. 406–418, Sep. 2019, doi: 10.1016/j.future.2019.02.008.

[10] D. Peshevski, V. Zdraveski, and S. Ristov, "Parallel near-duplicate document detection using general-purpose GPU," *Computing and Informatics*, vol. 43, no. 3, pp. 583–610, 2024, doi: 10.31577/cai_2024_3_583.

[11] T. Niemi, J. P. Ghobril, G. Defossez, S. Germann, E. Martin, and J.-L. Bulliard, "Automatic (near-) duplicate content document detection in a cancer registry," *International Journal of Medical Informatics*, vol. 195, Art. no. 105799, Mar. 2025, doi: 10.1016/j.ijmedinf.2025.105799.